

Mathematics for Machine Learning

Marc Deisenroth

Statistical Machine Learning Group
Department of Computing
Imperial College London

 @mpd37
m.deisenroth@imperial.ac.uk
marc@prowler.io

Deep Learning Indaba
University of the Witwatersrand
Johannesburg, South Africa

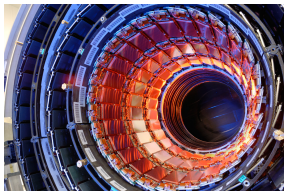
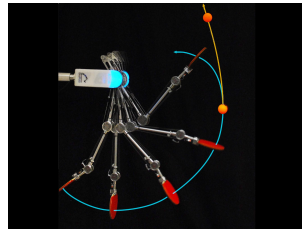
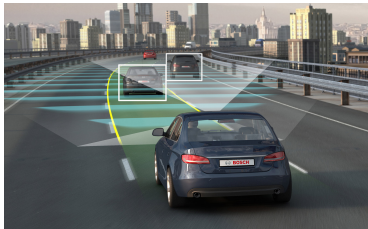
September 10, 2017

Applications of Machine Learning

lidar + car



leopard



Today's Recommendations For You

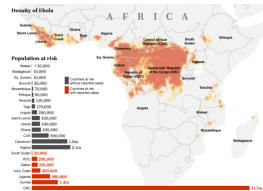
Here's a daily sample of items recommended for you. Click here to [see all recommendations](#)

Super-Fastest Web-Scale Algorithms (Paperback) by Steve Seiden
★★★★★ (7) \$23.10
Fix this recommendation

Simply Inverse (Paperback) by Kevin Van
★★★★★ (1) \$26.37
Fix this recommendation

The Art & Science of Java (Paperback)
★★★★★ (1) \$26.37
Fix this recommendation

Any Category Algorithms Boxed Sets Business & Culture Java Graphic Design Microsoft Networking Networks, Protocols & APIs New SQL



Mathematical Concepts in Machine Learning



- ▶ Linear algebra and matrix decomposition
- ▶ Differentiation
- ▶ Optimization
- ▶ Integration
- ▶ Probability theory and Bayesian inference
- ▶ Functional analysis - statistical

Outline

Introduction

Differentiation

Integration

Overview

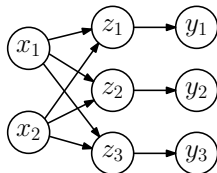
Introduction

Differentiation

Integration

Feedforward Neural Network

$$y = \sigma(z)$$
$$z = Ax + b$$



$$y_1 = \sigma(z_1)$$
$$y_2 = \sigma(z_2)$$

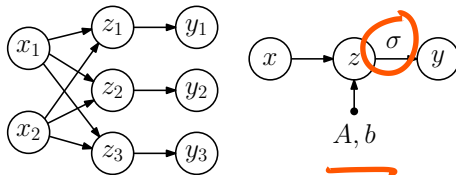
$$z_1 = a_1 x_1 + a_2 x_2 + b_1$$

Feed

$$y_1 = \sigma(a_1 x_1 + a_2 x_2 + b_1)$$

Feedforward Neural Network

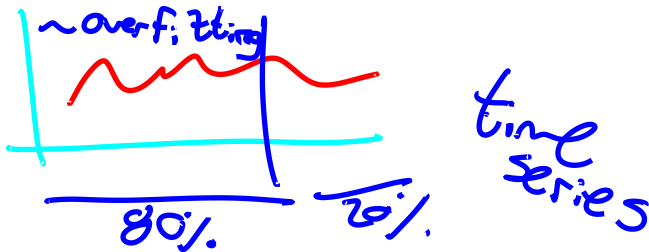
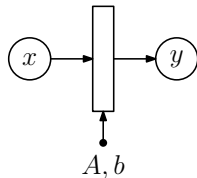
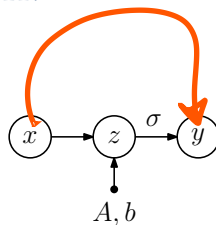
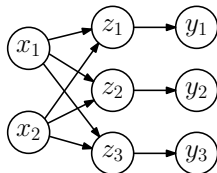
$$\mathbf{y} = \sigma(\mathbf{z})$$
$$\mathbf{z} = \mathbf{Ax} + \mathbf{b}$$



Feedforward Neural Network

$$y = \sigma(z)$$

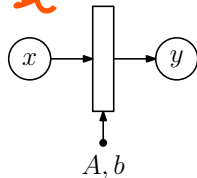
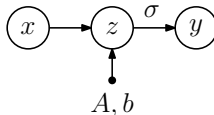
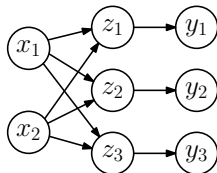
$$z = Ax + b$$



Feedforward Neural Network

$$y = \sigma(z)$$

$$z = Ax + b$$



Training a neural network means parameter optimization:
Typically via some form of gradient descent

► **Challenge 1: Differentiation.** Compute gradients of a loss function with respect to neural network parameters A, b



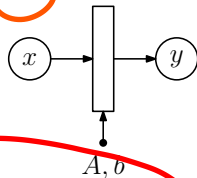
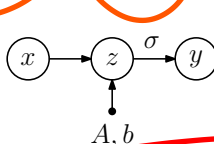
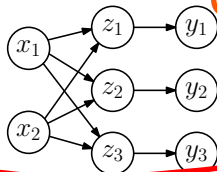
$$\left[\begin{array}{c} \frac{\partial y}{\partial z} \\ \frac{\partial z}{\partial x} \end{array} \right] = \text{gradient}$$

Feedforward Neural Network

$$z_j = a_1 x_1 + a_2 x_2 + b_j$$

$$y = \sigma(z)$$

$$z = Ax + b$$



- ▶ Training a neural network means parameter optimization:
Typically via some form of gradient descent

► **Challenge 1: Differentiation.** Compute gradients of a loss function with respect to neural network parameters A, b

- ▶ Computing statistics (e.g., means, variances) of predictions

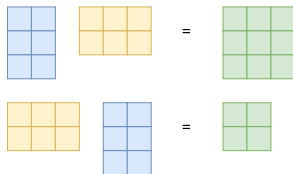
► **Challenge 2: Integration** Propagate uncertainty through a neural network

Rec ΔC

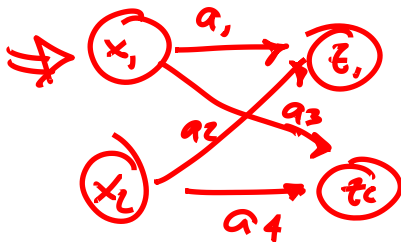
Learning

Background: Matrix Multiplication

- Matrix multiplication is not commutative, i.e., $AB \neq BA$



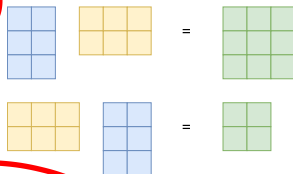
$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix}$$



Background: Matrix Multiplication

params = vector (neurons)
weights = vector
matrix

- Matrix multiplication is not commutative, i.e., $AB \neq BA$

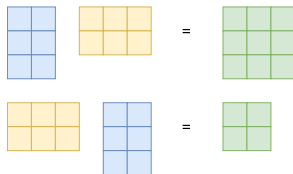


- When multiplying matrices, the “neighboring” dimensions have to fit:

$$\underbrace{A}_{n \times k} \underbrace{B}_{k \times m} = \underbrace{C}_{n \times m}$$

Background: Matrix Multiplication

- Matrix multiplication is not commutative, i.e., $AB \neq BA$



- When multiplying matrices, the “neighboring” dimensions have to fit:

$$\underbrace{A}_{n \times k} \underbrace{B}_{k \times m} = \underbrace{C}_{n \times m}$$

$$y = Ax$$

$$y_i = \sum_j A_{ij} x_j$$

$$C = AB$$

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

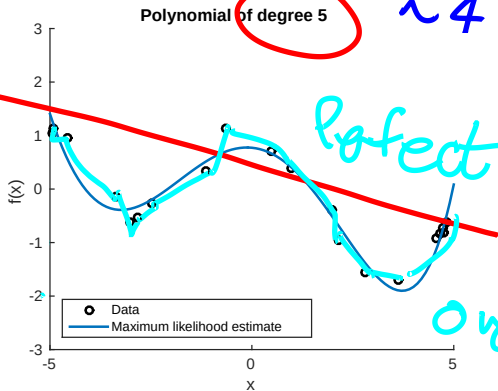
$$y = A \cdot \text{dot}(x)$$

$$y = \text{np.einsum}('ij, j', A, x)$$

$$C = A \cdot \text{dot}(B)$$

$$C = \text{np.einsum}('ik, kj', A, B)$$

Curve Fitting (Regression) in Machine Learning (1)



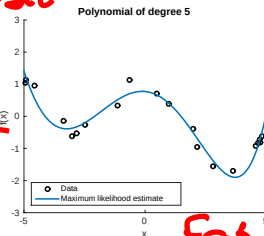
- ▶ Setting: Given inputs x , predict outputs/targets y
- ▶ Model f that depends on parameters θ . Examples:
 - ▶ Linear model: $f(x, \theta) = \theta^\top x$, $x, \theta \in \mathbb{R}^D$
 - ▶ Neural network: $f(x, \theta) = NN(x, \theta)$

Curve Fitting (Regression) in Machine Learning (2)

- ▶ Training data, e.g., N pairs (x_i, y_i) of inputs x_i and observations y_i

- ▶ Training the model means finding parameters θ^* , such that $f(x_i, \theta^*) \approx y_i$

test



Forces

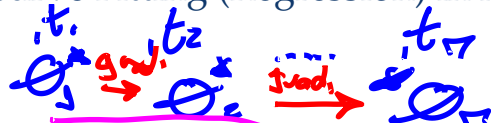
y
 y_{pred}

$f \rightarrow \text{model}$

y

diff $\Rightarrow$ error

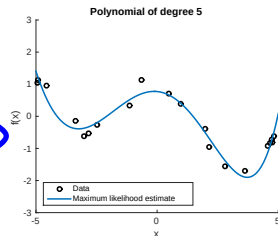
Curve Fitting (Regression) in Machine Learning (2)



less error

- ▶ Training data, e.g., N pairs (x_i, y_i) of inputs x_i and observations y_i

- ▶ Training the model means finding parameters θ^* , such that $f(x_i, \theta^*) \approx y_i$



- ▶ Define a loss function, e.g., $\sum_{i=1}^N (y_i - f(x_i, \theta))^2$, which we want to optimize
error
- ▶ Typically: Optimization based on some form of gradient descent
▶ Differentiation required

Overview

Introduction

Differentiation

Integration

Differentiation: Outline

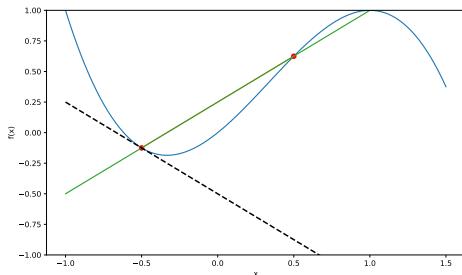
1. Scalar differentiation: $f : \mathbb{R} \rightarrow \mathbb{R}$
2. Multivariate case: $f : \mathbb{R}^N \rightarrow \mathbb{R}$
3. Vector fields: $f : \mathbb{R}^N \rightarrow \mathbb{R}^M$
4. General derivatives: $f : \mathbb{R}^{M \times N} \rightarrow \mathbb{R}^{P \times Q}$

Scalar Differentiation $f : \mathbb{R} \rightarrow \mathbb{R}$

- Derivative defined as the limit of the difference quotient

$$f'(x) = \frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- Slope of the secant line through $f(x)$ and $f(x+h)$



Examples

$$\begin{aligned}f(x) &= x^n \\f(x) &= \sin(x) \\f(x) &= \tanh(x) \\f(x) &= \exp(x) \\f(x) &= \log(x)\end{aligned}$$

$$\begin{aligned}f'(x) &= nx^{n-1} \\f'(x) &= \cos(x) \\f'(x) &= 1 - \tanh^2(x) \\f'(x) &= \exp(x) \\f'(x) &= \frac{1}{x}\end{aligned}$$

Rules

► Sum Rule

$$(f(x) + g(x))' = f'(x) + g'(x) = \frac{df}{dx} + \frac{dg}{dx}$$

Rules

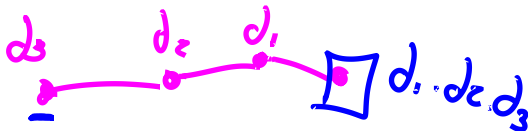
- Sum Rule

$$(f(x) + g(x))' = f'(x) + g'(x) = \frac{df}{dx} + \frac{dg}{dx}$$

- Product Rule

$$(f(x)g(x))' = f'(x)g(x) + f(x)g'(x) = \frac{df}{dx}g(x) + f(x)\frac{dg}{dx}$$

Rules



► Sum Rule

$$(f(x) + g(x))' = f'(x) + g'(x) = \frac{df}{dx} + \frac{dg}{dx}$$

► Product Rule

$$(f(x)g(x))' = f'(x)g(x) + f(x)g'(x) = \frac{df}{dx}g(x) + f(x)\frac{dg}{dx}$$

► Chain Rule

ANN

$$(g \circ f)'(x) = (g(f(x)))' = \underline{g'}(f(x))\underline{f'}(x) = \boxed{\frac{dg}{df} \frac{df}{dx}}$$

Example: Chain Rule

$$(g \circ f)'(x) = (g(f(x)))' = g'(f(x))f'(x) = \frac{dg}{df} \frac{df}{dx}$$

$g(z) = \tanh(z)$ activation function
 $z = f(x) = x^n$
 $(g \circ f)'(x) =$

Example: Chain Rule

$$(g \circ f)'(x) = (g(f(x)))' = g'(f(x))f'(x) = \frac{dg}{df} \frac{df}{dx} \dots$$

$$g(z) = \tanh(z)$$

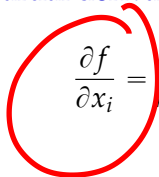
$$z = f(x) = x^n$$

$$(g \circ f)'(x) = \underbrace{(1 - \tanh^2(x^n))}_{dg/df} \underbrace{nx^{n-1}}_{df/dx}$$

$$f : \mathbb{R}^N \rightarrow \mathbb{R}$$

$$y = f(\mathbf{x}), \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \in \mathbb{R}^N$$


- **Partial derivative** (change one coordinate at a time):


$$\frac{\partial f}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{i-1}, x_i + h, x_{i+1}, \dots, x_N) - f(\mathbf{x})}{h}$$

$$f : \mathbb{R}^N \rightarrow \mathbb{R}$$

$$y = f(\mathbf{x}), \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \in \mathbb{R}^N$$

- **Partial derivative** (change one coordinate at a time):

$$\frac{\partial f}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{i-1}, x_i + h, x_{i+1}, \dots, x_N) - f(\mathbf{x})}{h}$$

Jacobian vector (gradient) collects all partial derivatives:

$$\frac{df}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \dots & \frac{\partial f}{\partial x_N} \end{bmatrix} \in \mathbb{R}^{1 \times N}$$

Note: This is a row vector.

Example

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}$$

$$f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3 \in \mathbb{R}$$

Example

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}$$
$$f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3 \in \mathbb{R}$$

► Partial derivatives:

$$\frac{\partial f(x_1, x_2)}{\partial x_1} = 2x_1 x_2 + x_2^3$$
$$\frac{\partial f(x_1, x_2)}{\partial x_2} = x_1^2 + 3x_1 x_2^2$$

Example

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}$$

$$f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3 \in \mathbb{R}$$

- Partial derivatives:

$$\begin{aligned}\frac{\partial f(x_1, x_2)}{\partial x_1} &= 2x_1 x_2 + x_2^3 \\ \frac{\partial f(x_1, x_2)}{\partial x_2} &= x_1^2 + 3x_1 x_2^2\end{aligned}$$

- Gradient:

$$\frac{df}{d\mathbf{x}} = \left[\frac{\partial f(x_1, x_2)}{\partial x_1} \quad \frac{\partial f(x_1, x_2)}{\partial x_2} \right] = \left[2x_1 x_2 + x_2^3 \quad x_1^2 + 3x_1 x_2^2 \right] \in \mathbb{R}^{1 \times 2}.$$

Rules

- ▶ Sum Rule

$$\frac{\partial}{\partial \mathbf{x}} (f(\mathbf{x}) + g(\mathbf{x})) = \frac{\partial f}{\partial \mathbf{x}} + \frac{\partial g}{\partial \mathbf{x}}$$

- ▶ Product Rule

$$\frac{\partial}{\partial \mathbf{x}} (f(\mathbf{x})g(\mathbf{x})) = \frac{\partial f}{\partial \mathbf{x}} g(\mathbf{x}) + f(\mathbf{x}) \frac{\partial g}{\partial \mathbf{x}}$$

- ▶ Chain Rule

$$\frac{\partial}{\partial \mathbf{x}} (g \circ f)(\mathbf{x}) = \frac{\partial}{\partial \mathbf{x}} (g(f(\mathbf{x}))) = \frac{\partial g}{\partial f} \frac{\partial f}{\partial \mathbf{x}}$$

Example: Chain Rule

- ▶ Consider the function

$$L(\mathbf{e}) = \frac{1}{2} \|\mathbf{e}\|^2 = \frac{1}{2} \mathbf{e}^\top \mathbf{e}$$
$$\mathbf{e} = \mathbf{y} - \mathbf{A}\mathbf{x}, \quad \mathbf{x} \in \mathbb{R}^N, \mathbf{A} \in \mathbb{R}^{M \times N}, \mathbf{e}, \mathbf{y} \in \mathbb{R}^M$$

- ▶ Compute $dL/d\mathbf{x}$. What is the dimension/size of $dL/d\mathbf{x}$?

Example: Chain Rule

- Consider the function

$$L(e) = \frac{1}{2} \|e\|^2 = \frac{1}{2} e^\top e$$

$$e = y - Ax, \quad x \in \mathbb{R}^N, A \in \mathbb{R}^{M \times N}, e, y \in \mathbb{R}^M$$

- Compute dL/dx . What is the dimension/size of dL/dx ?
- $dL/dx \in \mathbb{R}^{1 \times N}$

$$\frac{dL}{dx} = \frac{dL}{de} \frac{de}{dx}$$
$$\frac{dL}{de} = e^\top \in \mathbb{R}^{1 \times M} \quad (1)$$

$$\frac{de}{dx} = -A \in \mathbb{R}^{M \times N} \quad (2)$$

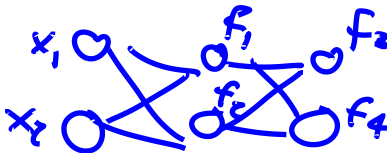
$$\Rightarrow \frac{dL}{dx} = e^\top (-A) = -(y - Ax)^\top A \in \mathbb{R}^{1 \times N}$$

$$f : \mathbb{R}^N \rightarrow \mathbb{R}^M$$

$$\mathbf{y} = f(\mathbf{x}) \in \mathbb{R}^M, \quad \mathbf{x} \in \mathbb{R}^N$$

$$\begin{bmatrix} y_1 \\ \vdots \\ y_M \end{bmatrix} = \begin{bmatrix} f_1(\mathbf{x}) \\ \vdots \\ f_M(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} f_1(x_1, \dots, x_N) \\ \vdots \\ f_M(x_1, \dots, x_N) \end{bmatrix}$$

$$f : \mathbb{R}^N \rightarrow \mathbb{R}^M$$



$$y = f(x) \in \mathbb{R}^M, \quad x \in \mathbb{R}^N$$

$$\begin{bmatrix} y_1 \\ \vdots \\ y_M \end{bmatrix} = \begin{bmatrix} f_1(x) \\ \vdots \\ f_M(x) \end{bmatrix} = \begin{bmatrix} f_1(x_1, \dots, x_N) \\ \vdots \\ f_M(x_1, \dots, x_N) \end{bmatrix}$$

- **Jacobian** matrix (collection of all partial derivatives)

$$\begin{bmatrix} \frac{dy_1}{dx} \\ \vdots \\ \frac{dy_M}{dx} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_N} \\ \vdots & & \vdots \\ \frac{\partial f_M}{\partial x_1} & \dots & \frac{\partial f_M}{\partial x_N} \end{bmatrix} \in \mathbb{R}^{M \times N}$$

Example

$$f(x) = \underline{Ax}, \quad f(x) \in \mathbb{R}^M, \quad A \in \mathbb{R}^{M \times N}, \quad x \in \mathbb{R}^N$$

- ▶ Compute the gradient df/dx
 - ▶ Dimension of df/dx :

Example

$$f(x) = Ax, \quad f(x) \in \mathbb{R}^M, \quad A \in \mathbb{R}^{M \times N}, \quad x \in \mathbb{R}^N$$

- Compute the gradient df/dx

- Dimension of df/dx :

Since $f: \mathbb{R}^N \rightarrow \mathbb{R}^M$, it follows that $df/dx \in \mathbb{R}^{M \times N}$

Example

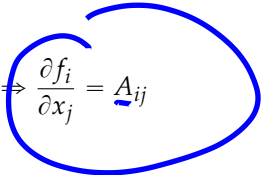
$$f(\mathbf{x}) = A\mathbf{x}, \quad f(\mathbf{x}) \in \mathbb{R}^M, \quad A \in \mathbb{R}^{M \times N}, \quad \mathbf{x} \in \mathbb{R}^N$$

- ▶ Compute the gradient $df/d\mathbf{x}$

- ▶ Dimension of $df/d\mathbf{x}$:

Since $f : \mathbb{R}^N \rightarrow \mathbb{R}^M$, it follows that $df/d\mathbf{x} \in \mathbb{R}^{M \times N}$

- ▶ Gradient:

$$f_i = \sum_{j=1}^N A_{ij}x_j \quad \Rightarrow \quad \frac{\partial f_i}{\partial x_j} = A_{ij}$$


(3)

Example

$$f(\mathbf{x}) = \mathbf{A}\mathbf{x}, \quad f(\mathbf{x}) \in \mathbb{R}^M, \quad \mathbf{A} \in \mathbb{R}^{M \times N}, \quad \mathbf{x} \in \mathbb{R}^N$$

- Compute the gradient $df/d\mathbf{x}$

- Dimension of $df/d\mathbf{x}$:

Since $f: \mathbb{R}^N \rightarrow \mathbb{R}^M$, it follows that $df/d\mathbf{x} \in \mathbb{R}^{M \times N}$

- Gradient:

$$\begin{aligned} f_i &= \sum_{j=1}^N A_{ij}x_j \quad \Rightarrow \quad \frac{\partial f_i}{\partial x_j} = A_{ij} \\ \Rightarrow \frac{df}{d\mathbf{x}} &= \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_N} \\ \vdots & & \vdots \\ \frac{\partial f_M}{\partial x_1} & \cdots & \frac{\partial f_M}{\partial x_N} \end{bmatrix} = \begin{bmatrix} A_{11} & \cdots & A_{1N} \\ \vdots & & \vdots \\ A_{M1} & \cdots & A_{MN} \end{bmatrix} = \mathbf{A} \quad (3) \end{aligned}$$

Chain Rule

$$\frac{\partial}{\partial \mathbf{x}}(g \circ f)(\mathbf{x}) = \frac{\partial}{\partial \mathbf{x}}(g(f(\mathbf{x}))) = \frac{\partial g}{\partial \mathbf{f}} \frac{\partial \mathbf{f}}{\partial \mathbf{x}}$$

Example

- Consider $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $\mathbf{x} : \mathbb{R} \rightarrow \mathbb{R}^2$

$$f(\mathbf{x}) = f(x_1, x_2) = x_1^2 + 2x_2,$$

$$\mathbf{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} \sin(t) \\ \cos(t) \end{bmatrix}$$

Example

- Consider $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $\mathbf{x} : \mathbb{R} \rightarrow \mathbb{R}^2$

$$f(\mathbf{x}) = f(x_1, x_2) = x_1^2 + 2x_2,$$

$$\mathbf{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} \sin(t) \\ \cos(t) \end{bmatrix}$$

- The dimensions $df/d\mathbf{x}$ and $d\mathbf{x}/dt$ are

Example

- Consider $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $\mathbf{x} : \mathbb{R} \rightarrow \mathbb{R}^2$

$$f(\mathbf{x}) = f(x_1, x_2) = x_1^2 + 2x_2,$$

$$\mathbf{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} \sin(t) \\ \cos(t) \end{bmatrix}$$

- The dimensions $df/d\mathbf{x}$ and $d\mathbf{x}/dt$ are 1×2 and 2×1 , respectively
- Compute the gradient df/dt using the chain rule.

Example



- Consider $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, $\mathbf{x} : \mathbb{R} \rightarrow \mathbb{R}^2$

$$f(\mathbf{x}) = f(x_1, x_2) = x_1^2 + 2x_2,$$

$$\mathbf{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} \sin(t) \\ \cos(t) \end{bmatrix}$$

- The dimensions $df/d\mathbf{x}$ and $d\mathbf{x}/dt$ are 1×2 and 2×1 , respectively
- Compute the gradient df/dt using the chain rule.

$$\begin{aligned} \frac{df}{dt} &= \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix} \begin{bmatrix} \frac{\partial x_1}{\partial t} \\ \frac{\partial x_2}{\partial t} \end{bmatrix} \\ &= \begin{bmatrix} 2 \sin t & 2 \end{bmatrix} \begin{bmatrix} \cos t \\ -\sin t \end{bmatrix} \\ &= 2 \sin t \cos t - 2 \sin t = 2 \sin t (\cos t - 1) \end{aligned}$$

BREAK

Derivatives with Matrices

- Re-cap: Gradient of a function $f : \mathbb{R}^D \rightarrow \mathbb{R}^E$ is an $E \times D$ -matrix:

target dimensions $\times$ # parameters

with

$$\frac{df}{d\mathbf{x}} \in \mathbb{R}^{E \times D}, \quad df[e, d] = \frac{\partial f_e}{\partial x_d}$$

Derivatives with Matrices

- ▶ Re-cap: Gradient of a function $f : \mathbb{R}^D \rightarrow \mathbb{R}^E$ is an $E \times D$ -matrix:

target dimensions $\times$ # parameters

with

$$\frac{df}{d\mathbf{x}} \in \mathbb{R}^{E \times D}, \quad df[e, d] = \frac{\partial f_e}{\partial x_d}$$

- ▶ Generalization to cases, where the parameters (D) or targets (E) are matrices, apply immediately

Derivatives with Matrices

- Re-cap: Gradient of a function $f : \mathbb{R}^D \rightarrow \mathbb{R}^E$ is an $E \times D$ -matrix:

target dimensions $\times$ # parameters

with

$$\frac{df}{d\mathbf{x}} \in \mathbb{R}^{E \times D}, \quad df[e, d] = \frac{\partial f_e}{\partial x_d}$$

- Generalization to cases, where the parameters (D) or targets (E) are matrices, apply immediately
- Assume $f : \mathbb{R}^{M \times N} \rightarrow \mathbb{R}^{P \times Q}$, then the gradient is a $(P \times Q) \times (M \times N)$ object (tensor) where

$$df[p, q, m, n] = \frac{\partial f_{pq}}{\partial X_{mn}}$$

Derivatives with Matrices: Example (1)

$$\mathbf{f} = \mathbf{A}\mathbf{x}, \quad \mathbf{f} \in \mathbb{R}^M, \mathbf{A} \in \mathbb{R}^{M \times N}, \mathbf{x} \in \mathbb{R}^N$$

Derivatives with Matrices: Example (1)

$$\mathbf{f} = \mathbf{A}\mathbf{x}, \quad \mathbf{f} \in \mathbb{R}^M, \mathbf{A} \in \mathbb{R}^{M \times N}, \mathbf{x} \in \mathbb{R}^N$$

$$\frac{d\mathbf{f}}{d\mathbf{A}} \in \mathbb{R}^{M \times (M \times N)}$$

$$\frac{d\mathbf{f}}{d\mathbf{A}} = \begin{bmatrix} \frac{\partial f_1}{\partial \mathbf{A}} \\ \vdots \\ \frac{\partial f_M}{\partial \mathbf{A}} \end{bmatrix}, \quad \frac{\partial f_i}{\partial \mathbf{A}} \in \mathbb{R}^{1 \times (M \times N)}$$

Derivatives with Matrices: Example (2)

$$f_i = \sum_{j=1}^N A_{ij} x_j, \quad i = 1, \dots, M$$

(4)

Derivatives with Matrices: Example (2)

$$f_i = \sum_{j=1}^N A_{ij} x_j, \quad i = 1, \dots, M$$

$$\frac{\partial f_i}{\partial A_{iq}} = x_q$$

(4)

Derivatives with Matrices: Example (2)

$$f_i = \sum_{j=1}^N A_{ij} x_j, \quad i = 1, \dots, M$$
$$\frac{\partial f_i}{\partial A_{iq}} = x_q \Rightarrow \frac{\partial f_i}{\partial A_{i,:}} = \mathbf{x}^\top \in \mathbb{R}^{1 \times 1 \times N}$$

(4)

Derivatives with Matrices: Example (2)

$$f_i = \sum_{j=1}^N A_{ij}x_j, \quad i = 1, \dots, M$$

$$\frac{\partial f_i}{\partial A_{iq}} = x_q \Rightarrow \frac{\partial f_i}{\partial A_{i,:}} = \mathbf{x}^\top \in \mathbb{R}^{1 \times 1 \times N}$$

$$\frac{\partial f_i}{\partial A_{k \neq i,:}} = \mathbf{0}^\top \in \mathbb{R}^{1 \times 1 \times N}$$

(4)

Derivatives with Matrices: Example (2)

$$f_i = \sum_{j=1}^N A_{ij} x_j, \quad i = 1, \dots, M$$

$$\frac{\partial f_i}{\partial A_{iq}} = x_q \Rightarrow \frac{\partial f_i}{\partial A_{i,:}} = \mathbf{x}^\top \in \mathbb{R}^{1 \times 1 \times N}$$

$$\frac{\partial f_i}{\partial A_{k \neq i,:}} = \mathbf{0}^\top \in \mathbb{R}^{1 \times 1 \times N}$$

$$\frac{\partial f_i}{\partial \mathbf{A}} = \begin{bmatrix} \mathbf{0}^\top \\ \vdots \\ \mathbf{x}^\top \\ \mathbf{0}^\top \\ \vdots \\ \mathbf{0}^\top \end{bmatrix} \in \mathbb{R}^{1 \times (M \times N)} \quad (4)$$

Example: Higher-Order Tensors



- Consider a matrix $A \in \mathbb{R}^{4 \times 2}$ whose entries depend on a vector $x \in \mathbb{R}^3$
- We can compute $dA(x)/dx \in \mathbb{R}^{4 \times 2 \times 3}$ in two equivalent ways:

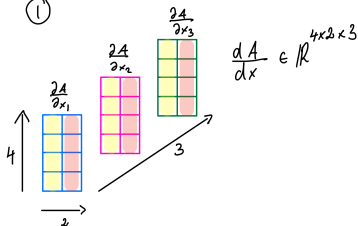
matrix
 $A \in \mathbb{R}^{4 \times 2}$



vector
 $x \in \mathbb{R}^3$



①



Example: Higher-Order Tensors

- Consider a matrix $A \in \mathbb{R}^{4 \times 2}$ whose entries depend on a vector $x \in \mathbb{R}^3$
- We can compute $dA(x)/dx \in \mathbb{R}^{4 \times 2 \times 3}$ in two equivalent ways:

matrix
 $A \in \mathbb{R}^{4 \times 2}$

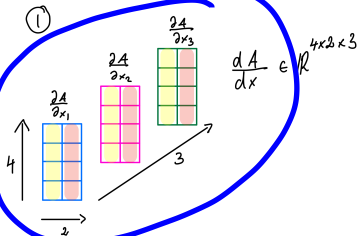


vector
 $x \in \mathbb{R}^3$

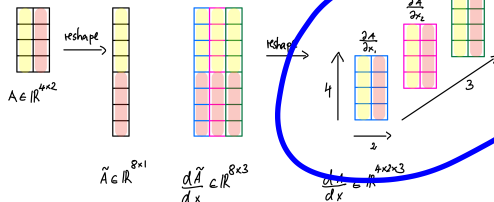


(dim, data)

①

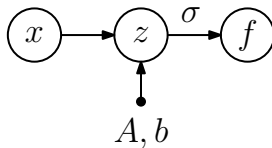


②



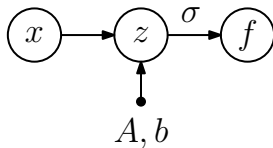
TPU

Gradients of a Single-Layer Neural Network (1)



$$f = \tanh(\underbrace{Ax + b}_{=: z \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad \underline{x} \in \mathbb{R}^N, \underline{A} \in \mathbb{R}^{M \times N}, \underline{b} \in \mathbb{R}^M$$

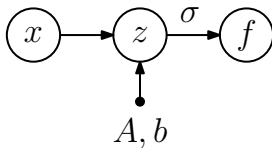
Gradients of a Single-Layer Neural Network (1)



$$f = \tanh(\underbrace{Ax + b}_{=:z \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad x \in \mathbb{R}^N, A \in \mathbb{R}^{M \times N}, b \in \mathbb{R}^M$$

$$\frac{\partial f}{\partial b} = \underbrace{\frac{\partial f}{\partial z}}_{M \times M} \underbrace{\frac{\partial z}{\partial b}}_{M \times M} \in \mathbb{R}^{M \times M}$$

Gradients of a Single-Layer Neural Network (1)



$$f = \tanh(\underbrace{Ax + b}_{=: z \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad x \in \mathbb{R}^N, A \in \mathbb{R}^{M \times N}, \mathbf{b} \in \mathbb{R}^M$$

$$\frac{\partial f}{\partial \mathbf{b}} = \underbrace{\frac{\partial f}{\partial \mathbf{z}}}_{M \times M} \underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{b}}}_{M \times M} \in \mathbb{R}^{M \times M}$$

$$\frac{\partial f}{\partial A} = \underbrace{\frac{\partial f}{\partial \mathbf{z}}}_{M \times M} \underbrace{\frac{\partial \mathbf{z}}{\partial A}}_{M \times (M \times N)} \in \mathbb{R}^{M \times (M \times N)}$$

Gradients of a Single-Layer Neural Network (2)

$$f = \tanh(\underbrace{Ax + b}_{z \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad x \in \mathbb{R}^N, A \in \mathbb{R}^{M \times N}, b \in \mathbb{R}^M$$

$$\frac{\partial f}{\partial b} = \underbrace{\frac{\partial f}{\partial z}}_{M \times M} \underbrace{\frac{\partial z}{\partial b}}_{M \times M} \in \mathbb{R}^{M \times M}$$

Gradients of a Single-Layer Neural Network (2)

$$f = \tanh(\underbrace{Ax + b}_{=:z \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad x \in \mathbb{R}^N, A \in \mathbb{R}^{M \times N}, b \in \mathbb{R}^M$$

$$\frac{\partial f}{\partial b} = \underbrace{\frac{\partial f}{\partial z}}_{M \times M} \underbrace{\frac{\partial z}{\partial b}}_{M \times M} \in \mathbb{R}^{M \times M}$$

$$\frac{\partial f}{\partial z} = \text{diag}(1 - \tanh^2(z)) \in \mathbb{R}^{M \times M}$$

Gradients of a Single-Layer Neural Network (2)

$$f = \tanh(\underbrace{Ax + b}_{=:z \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad x \in \mathbb{R}^N, A \in \mathbb{R}^{M \times N}, b \in \mathbb{R}^M$$

$$\frac{\partial f}{\partial b} = \underbrace{\frac{\partial f}{\partial z}}_{M \times M} \underbrace{\frac{\partial z}{\partial b}}_{M \times M} \in \mathbb{R}^{M \times M}$$

$$\frac{\partial f}{\partial z} = \text{diag}(1 - \tanh^2(z)) \in \mathbb{R}^{M \times M}$$

$$\frac{\partial z}{\partial b} = \underline{I} \in \mathbb{R}^{M \times M}$$

(5)

$$\frac{\partial f}{\partial b}[i, j] = \sum_{l=1}^M \frac{\partial f}{\partial z}[i, l] \frac{\partial z}{\partial b}[l, j]$$

Gradients of a Single-Layer Neural Network (2)

$$f = \tanh(\underbrace{Ax + b}_{=:z \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad x \in \mathbb{R}^N, A \in \mathbb{R}^{M \times N}, b \in \mathbb{R}^M$$

$$\frac{\partial f}{\partial b} = \underbrace{\frac{\partial f}{\partial z}}_{M \times M} \underbrace{\frac{\partial z}{\partial b}}_{M \times M} \in \mathbb{R}^{M \times M}$$

$$\frac{\partial f}{\partial z} = \text{diag}(1 - \tanh^2(z)) \in \mathbb{R}^{M \times M}$$

$$\frac{\partial z}{\partial b} = \mathbf{I} \in \mathbb{R}^{M \times M} \quad (5)$$

$$\frac{\partial f}{\partial b}[i, j] = \sum_{l=1}^M \frac{\partial f}{\partial z}[i, l] \frac{\partial z}{\partial b}[l, j]$$

$$\text{dfd}b = \text{np.einsum}('il, lj', \text{dfd}z, \text{dzdb})$$

Gradients of a Single-Layer Neural Network (3)

$$f = \tanh(\underbrace{\mathbf{A}\mathbf{x} + \mathbf{b}}_{=: \mathbf{z} \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad \mathbf{x} \in \mathbb{R}^N, \mathbf{A} \in \mathbb{R}^{M \times N}, \mathbf{b} \in \mathbb{R}^M$$

$$\frac{\partial f}{\partial \mathbf{A}} = \underbrace{\frac{\partial f}{\partial \mathbf{z}}}_{M \times M} \underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{A}}}_{M \times (M \times N)} \in \mathbb{R}^{M \times (M \times N)}$$

Gradients of a Single-Layer Neural Network (3)

$$\begin{aligned} f &= \tanh(\underbrace{\mathbf{A}\mathbf{x} + \mathbf{b}}_{=: \mathbf{z} \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad \mathbf{x} \in \mathbb{R}^N, \mathbf{A} \in \mathbb{R}^{M \times N}, \mathbf{b} \in \mathbb{R}^M \\ \frac{\partial f}{\partial \mathbf{A}} &= \underbrace{\frac{\partial f}{\partial \mathbf{z}}}_{M \times M} \underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{A}}}_{M \times (M \times N)} \in \mathbb{R}^{M \times (M \times N)} \\ \frac{\partial f}{\partial \mathbf{z}} &= \text{diag}(1 - \tanh^2(\mathbf{z})) \in \mathbb{R}^{M \times M} \end{aligned} \tag{6}$$

Gradients of a Single-Layer Neural Network (3)

$$f = \tanh(\underbrace{\mathbf{A}\mathbf{x} + \mathbf{b}}_{=: \mathbf{z} \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad \mathbf{x} \in \mathbb{R}^N, \mathbf{A} \in \mathbb{R}^{M \times N}, \mathbf{b} \in \mathbb{R}^M$$

$$\frac{\partial f}{\partial \mathbf{A}} = \underbrace{\frac{\partial f}{\partial \mathbf{z}}}_{M \times M} \underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{A}}}_{M \times (M \times N)} \in \mathbb{R}^{M \times (M \times N)}$$

$$\frac{\partial f}{\partial \mathbf{z}} = \text{diag}(1 - \tanh^2(\mathbf{z})) \in \mathbb{R}^{M \times M} \quad (6)$$

$\frac{\partial \mathbf{z}}{\partial \mathbf{A}} \quad \gg \text{See (4)}$

$$\frac{\partial f}{\partial \mathbf{A}}[i, j, k] = \sum_{l=1}^M \frac{\partial f}{\partial \mathbf{z}}[i, l] \frac{\partial \mathbf{z}}{\partial \mathbf{A}}[l, j, k]$$

Gradients of a Single-Layer Neural Network (3)

$$f = \tanh(\underbrace{\mathbf{A}\mathbf{x} + \mathbf{b}}_{=: \mathbf{z} \in \mathbb{R}^M}) \in \mathbb{R}^M, \quad \mathbf{x} \in \mathbb{R}^N, \mathbf{A} \in \mathbb{R}^{M \times N}, \mathbf{b} \in \mathbb{R}^M$$

$$\frac{\partial f}{\partial \mathbf{A}} = \underbrace{\frac{\partial f}{\partial \mathbf{z}}}_{M \times M} \underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{A}}}_{M \times (M \times N)} \in \mathbb{R}^{M \times (M \times N)}$$

$$\frac{\partial f}{\partial \mathbf{z}} = \text{diag}(1 - \tanh^2(\mathbf{z})) \in \mathbb{R}^{M \times M} \quad (6)$$

$\frac{\partial \mathbf{z}}{\partial \mathbf{A}} \quad \gg \text{See (4)}$

$$\frac{\partial f}{\partial \mathbf{A}}[i, j, k] = \sum_{l=1}^M \frac{\partial f}{\partial \mathbf{z}}[i, l] \frac{\partial \mathbf{z}}{\partial \mathbf{A}}[l, j, k]$$

$$\text{dfdA} = \text{np.einsum}('il, ljk', \text{dfdZ}, \text{dzdA})$$

Putting Things Together

- Inputs x , observed outputs $y = f(z, \theta) + \epsilon, \epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma)$

Putting Things Together

- Inputs x , observed outputs $y = f(z, \theta) + \epsilon$, $\epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma)$
- Train single-layer neural network with

$$f(z, \theta) = \tanh(z), \quad z = Ax + b, \quad \theta = \{A, b\}$$

Putting Things Together

- ▶ Inputs x , observed outputs $y = f(z, \theta) + \epsilon, \epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma)$
- ▶ Train single-layer neural network with

$$f(z, \theta) = \tanh(z), \quad z = Ax + b, \quad \theta = \{A, b\}$$

- ▶ Find A, b , such that the squared loss

$$L(\theta) = \frac{1}{2} \|e\|^2, \quad e = y - f(z, \theta)$$

is minimized

Putting Things Together

- Inputs x , observed outputs $y = f(z, \theta) + \epsilon, \epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma)$
- Train single-layer neural network with

$$f(z, \theta) = \tanh(z), \quad z = Ax + b, \quad \theta = \{A, b\}$$

- Find A, b , such that the squared loss

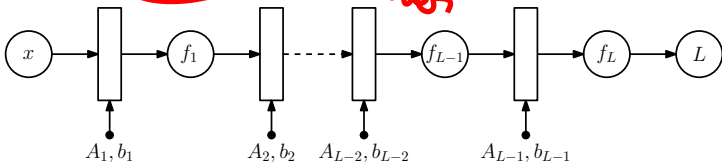
$$L(\theta) = \frac{1}{2} \|e\|^2, \quad e = y - f(z, \theta)$$

is minimized

- Partial derivatives:

$$\left. \begin{aligned} \frac{\partial L}{\partial A} &= \frac{\partial L}{\partial e} \frac{\partial e}{\partial f} \frac{\partial f}{\partial z} \frac{\partial z}{\partial A} \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e} \frac{\partial e}{\partial f} \frac{\partial f}{\partial z} \frac{\partial z}{\partial b} \end{aligned} \right\} \begin{aligned} \frac{\partial L}{\partial e} &\gg (1) & \frac{\partial e}{\partial f} &\gg (2), (3) & \frac{\partial f}{\partial z} &\gg (6) \\ \frac{\partial z}{\partial A} &\gg (4) & \frac{\partial z}{\partial b} &\gg (5) \end{aligned}$$

Gradients of a Multi-Layer Neural Network

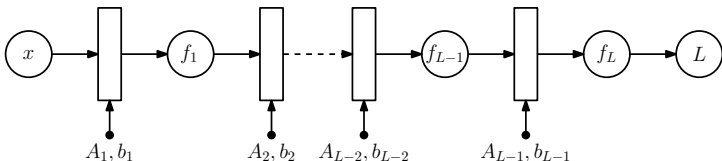


- Inputs x observed outputs y
- Train multi-layer neural network with

$$f_0 = x$$

$$f_i = \sigma_i(A_{i-1}f_{i-1} + b_{i-1}), \quad i = 1, \dots, L$$

Gradients of a Multi-Layer Neural Network



- ▶ Inputs x , observed outputs y
- ▶ Train multi-layer neural network with

$$f_0 = x$$

$$f_i = \sigma_i(A_{i-1}f_{i-1} + b_{i-1}), \quad i = 1, \dots, L$$

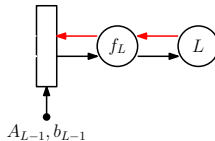
- ▶ Find A_j, b_j for $j = 0, \dots, L-1$, such that the squared loss

$$L(\theta) = \|y - f_L(\theta, x)\|^2$$

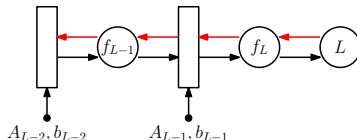
is minimized, where $\theta = \{A_j, b_j\}$, $j = 0, \dots, L-1$

Gradients of a Multi-Layer Neural Network

$$\frac{\partial L}{\partial \theta_{L-1}} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial \theta_{L-1}}$$



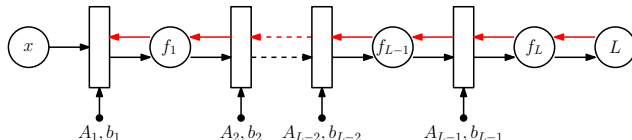
Gradients of a Multi-Layer Neural Network



$$\frac{\partial L}{\partial \theta_{L-1}} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial \theta_{L-1}}$$

$$\frac{\partial L}{\partial \theta_{L-2}} = \frac{\partial L}{\partial f_L} \left[\frac{\partial f_L}{\partial f_{L-1}} \frac{\partial f_{L-1}}{\partial \theta_{L-2}} \right]$$

Gradients of a Multi-Layer Neural Network

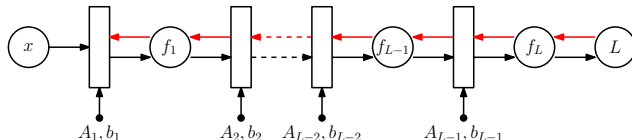


$$\frac{\partial L}{\partial \theta_{L-1}} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial \theta_{L-1}}$$

$$\frac{\partial L}{\partial \theta_{L-2}} = \frac{\partial L}{\partial f_L} \left[\frac{\partial f_L}{\partial f_{L-1}} \frac{\partial f_{L-1}}{\partial \theta_{L-2}} \right]$$

$$\frac{\partial L}{\partial \theta_{L-3}} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial f_{L-1}} \left[\frac{\partial f_{L-1}}{\partial f_{L-2}} \frac{\partial f_{L-2}}{\partial \theta_{L-3}} \right]$$

Gradients of a Multi-Layer Neural Network



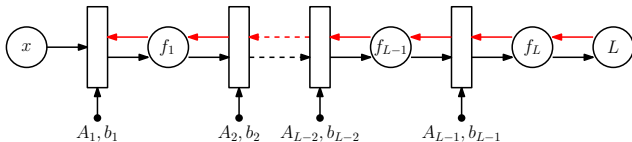
$$\frac{\partial L}{\partial \theta_{L-1}} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial \theta_{L-1}}$$

$$\frac{\partial L}{\partial \theta_{L-2}} = \frac{\partial L}{\partial f_L} \left[\frac{\partial f_L}{\partial f_{L-1}} \frac{\partial f_{L-1}}{\partial \theta_{L-2}} \right]$$

$$\frac{\partial L}{\partial \theta_{L-3}} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial f_{L-1}} \left[\frac{\partial f_{L-1}}{\partial f_{L-2}} \frac{\partial f_{L-2}}{\partial \theta_{L-3}} \right]$$

$$\frac{\partial L}{\partial \theta_i} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial f_{L-1}} \dots \left[\frac{\partial f_{i+2}}{\partial f_{i+1}} \frac{\partial f_{i+1}}{\partial \theta_i} \right]$$

Gradients of a Multi-Layer Neural Network



$$\frac{\partial L}{\partial \theta_{L-1}} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial \theta_{L-1}}$$

$$\frac{\partial L}{\partial \theta_{L-2}} = \frac{\partial L}{\partial f_L} \left[\frac{\partial f_L}{\partial f_{L-1}} \frac{\partial f_{L-1}}{\partial \theta_{L-2}} \right]$$

$$\frac{\partial L}{\partial \theta_{L-3}} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial f_{L-1}} \left[\frac{\partial f_{L-1}}{\partial f_{L-2}} \frac{\partial f_{L-2}}{\partial \theta_{L-3}} \right]$$

$$\frac{\partial L}{\partial \theta_i} = \frac{\partial L}{\partial f_L} \frac{\partial f_L}{\partial f_{L-1}} \dots \left[\frac{\partial f_{i+2}}{\partial f_{i+1}} \frac{\partial f_{i+1}}{\partial \theta_i} \right]$$

►► More details (including efficient implementation) later this week

Training Neural Networks as Maximum Likelihood Estimation

- ▶ Training a neural network in the above way corresponds to **maximum likelihood estimation**:
 - ▶ If $\mathbf{y} = \text{NN}(\mathbf{x}, \boldsymbol{\theta}) + \epsilon$, $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ then the **log-likelihood** is

$$\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) = -\frac{1}{2} \|\mathbf{y} - \text{NN}(\mathbf{x}, \boldsymbol{\theta})\|^2$$

Training Neural Networks as Maximum Likelihood Estimation

- ▶ Training a neural network in the above way corresponds to **maximum likelihood estimation**:

- ▶ If $\mathbf{y} = NN(\mathbf{x}, \boldsymbol{\theta}) + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ then the **log-likelihood** is

$$\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) = -\frac{1}{2} \|\mathbf{y} - NN(\mathbf{x}, \boldsymbol{\theta})\|^2$$

- ▶ Find $\boldsymbol{\theta}^*$ by **minimizing the negative log-likelihood**:

$$\begin{aligned}\boldsymbol{\theta}^* &= \arg \min_{\boldsymbol{\theta}} -\log p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) \\ &= \arg \min_{\boldsymbol{\theta}} \frac{1}{2} \|\mathbf{y} - NN(\mathbf{x}, \boldsymbol{\theta})\|^2 \\ &= \arg \min_{\boldsymbol{\theta}} L(\boldsymbol{\theta})\end{aligned}$$

Training Neural Networks as Maximum Likelihood Estimation

- ▶ Training a neural network in the above way corresponds to **maximum likelihood estimation**:

- ▶ If $\mathbf{y} = NN(\mathbf{x}, \boldsymbol{\theta}) + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ then the **log-likelihood** is

$$\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) = -\frac{1}{2}\|\mathbf{y} - NN(\mathbf{x}, \boldsymbol{\theta})\|^2$$

- ▶ Find $\boldsymbol{\theta}^*$ by **minimizing the negative log-likelihood**:

$$\begin{aligned}\boldsymbol{\theta}^* &= \arg \min_{\boldsymbol{\theta}} -\log p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) \\ &= \arg \min_{\boldsymbol{\theta}} \frac{1}{2}\|\mathbf{y} - NN(\mathbf{x}, \boldsymbol{\theta})\|^2 \\ &= \arg \min_{\boldsymbol{\theta}} L(\boldsymbol{\theta})\end{aligned}$$

- ▶ Maximum likelihood estimation can lead to **overfitting** (interpret noise as signal)

Example: Linear Regression (1)

- ▶ Linear regression with a polynomial of order M :

$$y = f(x, \boldsymbol{\theta}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$$

$$f(x, \boldsymbol{\theta}) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_M x^M = \sum_{i=0}^M \theta_i x^i$$

Example: Linear Regression (1)

- ▶ Linear regression with a polynomial of order M :

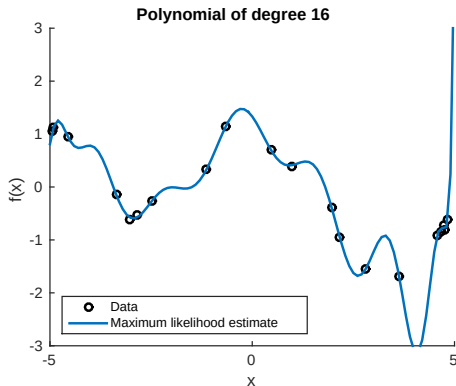
$$y = f(x, \boldsymbol{\theta}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$$

$$f(x, \boldsymbol{\theta}) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_M x^M = \sum_{i=0}^M \theta_i x^i$$

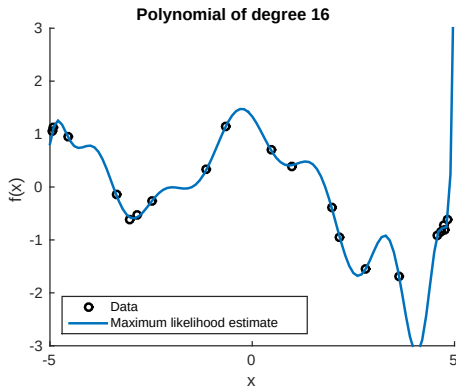
- ▶ Given inputs x_i and corresponding (noisy) observations y_i , $i = 1, \dots, N$, find parameters $\boldsymbol{\theta} = [\theta_0, \dots, \theta_M]^\top$, that minimize the squared loss (equivalently: maximize the likelihood)

$$L(\boldsymbol{\theta}) = \sum_{i=1}^N (y_i - f(x_i, \boldsymbol{\theta}))^2$$

Example: Linear Regression (2)



Example: Linear Regression (2)



- ▶ Regularization, model selection etc. can address overfitting
 - ▶▶ Tutorials later this week
- ▶ Alternative approach based on [integration](#)

Overview

Introduction

Differentiation

Integration

Integration: Outline

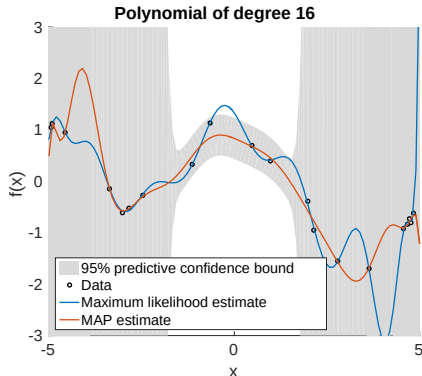
1. Motivation
2. Monte-Carlo estimation
3. Basic sampling algorithms

Bayesian Integration to Avoid Overfitting

- ▶ Instead of fitting a single set of parameters θ^* we can average over all plausible parameters

▶ Bayesian integration:

$$p(y|x) = \int p(y|x, \theta) p(\theta) d\theta$$



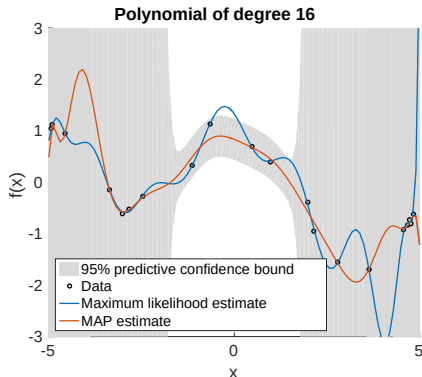
Bayesian Integration to Avoid Overfitting

- ▶ Instead of fitting a single set of parameters θ^* , we can average over all plausible parameters

▶▶ Bayesian integration:

$$p(y|x) = \int p(y|x, \theta) p(\theta) d\theta$$

- ▶ More details on what $p(\theta)$ is ▶▶ Tutorials later this week

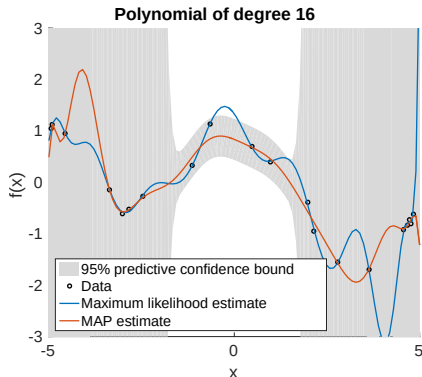


Bayesian Integration to Avoid Overfitting

- ▶ Instead of fitting a single set of parameters θ^* , we can average over all plausible parameters

▶▶ Bayesian integration:

$$p(y|x) = \int p(y|x, \theta) p(\theta) d\theta$$



- ▶ More details on what $p(\theta)$ is ▶▶ Tutorials later this week
- ▶ For neural networks this integration is intractable
- ▶▶ Approximations

Computing Statistics of Random Variables

- ▶ Computing means/(co)variances also requires solving integrals:

$$\mathbb{E}_x[\mathbf{x}] = \int \mathbf{x} p(\mathbf{x}) d\mathbf{x} =: \boldsymbol{\mu}_x$$

$$\mathbb{V}_x[\mathbf{x}] = \int (\mathbf{x} - \boldsymbol{\mu}_x)(\mathbf{x} - \boldsymbol{\mu}_x)^\top d\mathbf{x}$$

$$\text{Cov}[\mathbf{x}, \mathbf{y}] = \iint (\mathbf{x} - \boldsymbol{\mu}_x)(\mathbf{y} - \boldsymbol{\mu}_y)^\top d\mathbf{x} d\mathbf{y}$$

Computing Statistics of Random Variables

- ▶ Computing means/(co)variances also requires solving integrals:

$$\mathbb{E}_x[\mathbf{x}] = \int \mathbf{x} p(\mathbf{x}) d\mathbf{x} =: \boldsymbol{\mu}_x$$

$$\mathbb{V}_x[\mathbf{x}] = \int (\mathbf{x} - \boldsymbol{\mu}_x)(\mathbf{x} - \boldsymbol{\mu}_x)^\top d\mathbf{x}$$

$$\text{Cov}[\mathbf{x}, \mathbf{y}] = \iint (\mathbf{x} - \boldsymbol{\mu}_x)(\mathbf{y} - \boldsymbol{\mu}_y)^\top d\mathbf{x} d\mathbf{y}$$

- ▶ These **integrals can often not be computed in closed form**
 - ▶▶ Approximations

Approximate Integration

- **Numerical integration** (low-dimensional problems)
- **Bayesian quadrature**, e.g., O'Hagan (1987, 1991); Rasmussen & Ghahramani (2003)
- **Variational Bayes**, e.g., Jordan et al. (1999)
- **Expectation Propagation**, Oppner & Winther (2001); Minka (2001)
- **Monte-Carlo Methods**, e.g., Gilks et al. (1996), Robert & Casella (2004), Bishop (2006)

Monte Carlo Methods—Motivation

- ▶ Monte Carlo methods are computational techniques that make use of random numbers
- ▶ Two typical problems:
 1. **Problem 1:** Generate samples $\{\mathbf{x}^{(s)}\}$ from a given probability distribution $p(\mathbf{x})$, e.g., for simulation or representations of data distributions

Monte Carlo Methods—Motivation

- ▶ Monte Carlo methods are computational techniques that make use of **random numbers**
- ▶ Two typical problems:
 1. **Problem 1:** **Generate samples** $\{\mathbf{x}^{(s)}\}$ from a given probability distribution $p(\mathbf{x})$, e.g., for simulation or representations of data distributions
 2. **Problem 2:** **Compute expectations** of functions under that distribution:

$$\mathbb{E}[f(\mathbf{x})] = \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x}$$

Monte Carlo Methods—Motivation

- ▶ Monte Carlo methods are computational techniques that make use of **random numbers**
- ▶ Two typical problems:
 1. **Problem 1:** **Generate samples** $\{\mathbf{x}^{(s)}\}$ from a given probability distribution $p(\mathbf{x})$, e.g., for simulation or representations of data distributions
 2. **Problem 2:** **Compute expectations** of functions under that distribution:

$$\mathbb{E}[f(\mathbf{x})] = \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x}$$

▶▶ Example: Means/variances of distributions, predictions

Complication: Integral cannot be evaluated analytically

Problem 2: Monte Carlo Estimation

- ▶ **Computing expectations** via statistical sampling:

$$\begin{aligned}\mathbb{E}[f(\mathbf{x})] &= \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x} \\ &\approx \frac{1}{S} \sum_{s=1}^S f(\mathbf{x}^{(s)}), \quad \mathbf{x}^{(s)} \sim p(\mathbf{x})\end{aligned}$$

Problem 2: Monte Carlo Estimation

- ▶ **Computing expectations** via statistical sampling:

$$\begin{aligned}\mathbb{E}[f(\mathbf{x})] &= \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x} \\ &\approx \frac{1}{S} \sum_{s=1}^S f(\mathbf{x}^{(s)}), \quad \mathbf{x}^{(s)} \sim p(\mathbf{x})\end{aligned}$$

- ▶ **Making predictions** (e.g., Bayesian regression with inputs $\mathbf{x}$ and targets $\mathbf{y}$)

$$\begin{aligned}p(\mathbf{y}|\mathbf{x}) &= \int p(\mathbf{y}|\boldsymbol{\theta}, \mathbf{x}) \underbrace{p(\boldsymbol{\theta})}_{\text{Parameter distribution}} d\boldsymbol{\theta} \\ &\approx \frac{1}{S} \sum_{s=1}^S p(\mathbf{y}|\boldsymbol{\theta}^{(s)}, \mathbf{x}), \quad \boldsymbol{\theta}^{(s)} \sim p(\boldsymbol{\theta})\end{aligned}$$

Problem 2: Monte Carlo Estimation

- ▶ **Computing expectations** via statistical sampling:

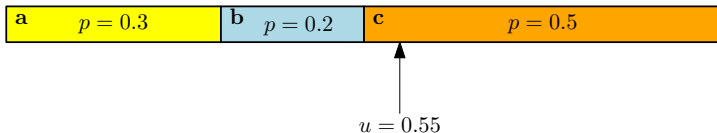
$$\begin{aligned}\mathbb{E}[f(\mathbf{x})] &= \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x} \\ &\approx \frac{1}{S} \sum_{s=1}^S f(\mathbf{x}^{(s)}), \quad \mathbf{x}^{(s)} \sim p(\mathbf{x})\end{aligned}$$

- ▶ **Making predictions** (e.g., Bayesian regression with inputs $\mathbf{x}$ and targets $\mathbf{y}$)

$$\begin{aligned}p(\mathbf{y}|\mathbf{x}) &= \int p(\mathbf{y}|\boldsymbol{\theta}, \mathbf{x}) \underbrace{p(\boldsymbol{\theta})}_{\text{Parameter distribution}} d\boldsymbol{\theta} \\ &\approx \frac{1}{S} \sum_{s=1}^S p(\mathbf{y}|\boldsymbol{\theta}^{(s)}, \mathbf{x}), \quad \boldsymbol{\theta}^{(s)} \sim p(\boldsymbol{\theta})\end{aligned}$$

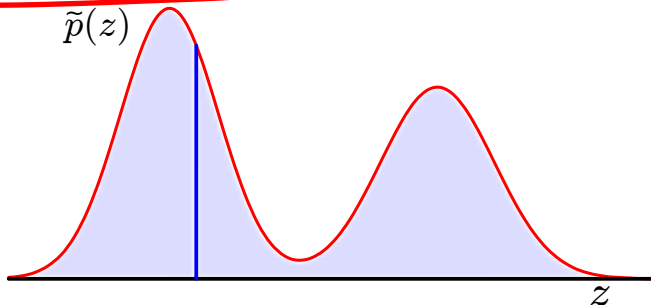
- ▶ **Key problem:** Generating samples from $p(\mathbf{x})$ or $p(\boldsymbol{\theta})$
▶▶ Need to solve **Problem 1**

Sampling Discrete Values



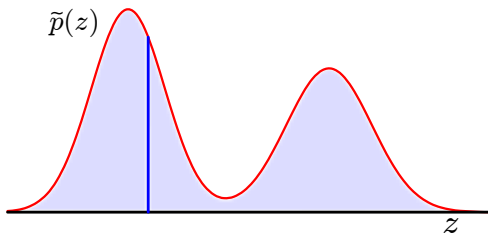
- ▶ $u \sim \mathcal{U}[0, 1]$, where $\mathcal{U}$ is the uniform distribution
- ▶ $u = 0.55 \Rightarrow x = c$

Continuous Variables



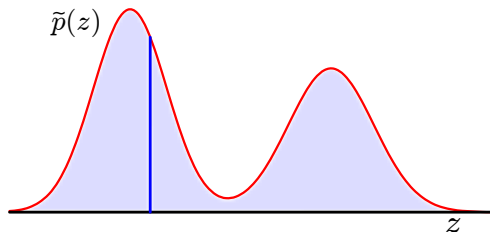
- More complicated
- Geometrically, we wish to sample uniformly from the area under the curve
- Two algorithms here:
 - Rejection sampling
 - Importance sampling

Rejection Sampling: Setting



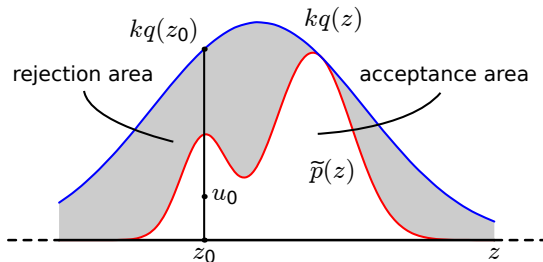
- Assume:
 - Sampling from $p(z)$ is difficult
 - Evaluating $\tilde{p}(z) = \frac{p(z)}{Z}$ is easy (and Z may be unknown)

Rejection Sampling: Setting



- ▶ Assume:
 - ▶ Sampling from $p(z)$ is difficult
 - ▶ Evaluating $\tilde{p}(z) = Zp(z)$ is easy (and Z may be unknown)
- ▶ Find a simpler distribution (**proposal distribution**) $q(z)$ from which we can easily draw samples (e.g., Gaussian, Laplace)
- ▶ Find an **upper bound** $kq(z) \geq \tilde{p}(z)$

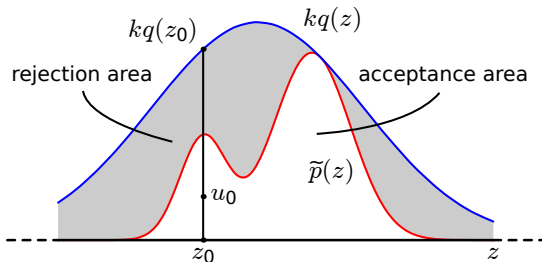
Rejection Sampling: Algorithm



Adapted from PRML (Bishop, 2006)

1. Generate $z_0 \sim q(z)$
2. Generate $u_0 \sim \mathcal{U}[0, kq(z_0)]$
3. If $u_0 > \tilde{p}(z_0)$, reject the sample. Otherwise, retain z_0

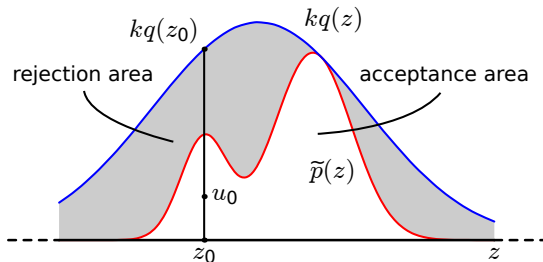
Properties



Adapted from PRML (Bishop, 2006)

- Accepted pairs (z, u) are uniformly distributed under $\tilde{p}(z)$

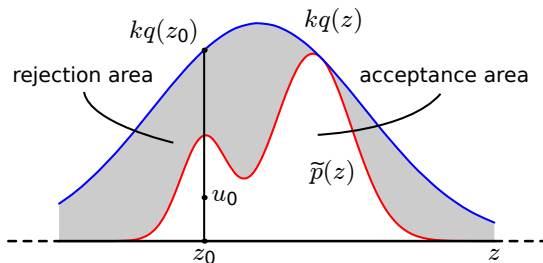
Properties



Adapted from PRML (Bishop, 2006)

- ▶ Accepted pairs (z, u) are uniformly distributed under $\tilde{p}(z)$
- ▶ Probability density of the z -coordinates of accepted points must be proportional to $p(z)$

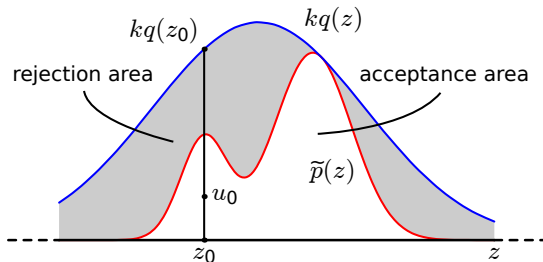
Properties



Adapted from PRML (Bishop, 2006)

- ▶ Accepted pairs (z, u) are uniformly distributed under $\tilde{p}(z)$
- ▶ Probability density of the z -coordinates of accepted points must be proportional to $p(z)$
- ▶ Samples are independent samples from $p(z)$

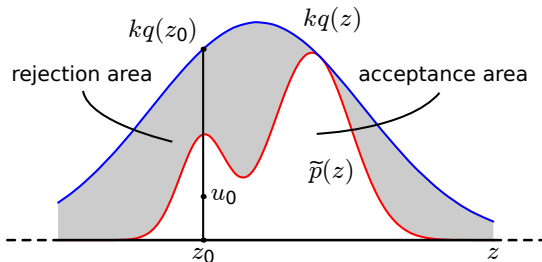
Shortcomings



Adapted from PRML (Bishop, 2006)

- Finding the upper bound k is tricky

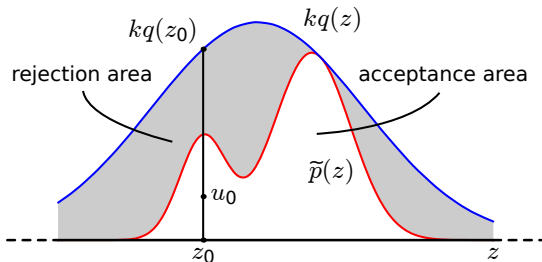
Shortcomings



Adapted from PRML (Bishop, 2006)

- ▶ Finding the upper bound k is tricky
- ▶ In high dimensions the factor k is probably huge

Shortcomings



Adapted from PRML (Bishop, 2006)

- ▶ Finding the upper bound k is tricky
- ▶ In high dimensions the factor k is probably huge
- ▶ **Low acceptance rate/high rejection rate** of samples

Importance Sampling

Key idea: Do not throw away all rejected samples, but give them lower weight by rewriting the integral as an expectation under a simpler distribution q (**proposal distribution**):

$$\mathbb{E}_p[f(\mathbf{x})] = \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x}$$

Importance Sampling

Key idea: Do not throw away all rejected samples, but give them lower weight by rewriting the integral as an expectation under a simpler distribution q (**proposal distribution**):

$$\begin{aligned}\mathbb{E}_p[f(\mathbf{x})] &= \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x} \\ &= \int f(\mathbf{x})p(\mathbf{x})\frac{q(\mathbf{x})}{q(\mathbf{x})}d\mathbf{x}\end{aligned}$$

Importance Sampling

Key idea: Do not throw away all rejected samples, but give them lower weight by rewriting the integral as an expectation under a simpler distribution q (**proposal distribution**):

$$\begin{aligned}\mathbb{E}_p[f(\mathbf{x})] &= \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x} \\ &= \int f(\mathbf{x})p(\mathbf{x})\frac{q(\mathbf{x})}{q(\mathbf{x})}d\mathbf{x} = \int f(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}q(\mathbf{x})d\mathbf{x}\end{aligned}$$

Importance Sampling

Key idea: Do not throw away all rejected samples, but give them lower weight by rewriting the integral as an expectation under a simpler distribution q (**proposal distribution**):

$$\begin{aligned}\mathbb{E}_p[f(\mathbf{x})] &= \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x} \\ &= \int f(\mathbf{x})p(\mathbf{x})\frac{q(\mathbf{x})}{q(\mathbf{x})}d\mathbf{x} = \int f(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}q(\mathbf{x})d\mathbf{x} \\ &= \mathbb{E}_q\left[f(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}\right]\end{aligned}$$

Importance Sampling

Key idea: Do not throw away all rejected samples, but give them lower weight by rewriting the integral as an expectation under a simpler distribution q (**proposal distribution**):

$$\begin{aligned}\mathbb{E}_p[f(\mathbf{x})] &= \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x} \\ &= \int f(\mathbf{x})p(\mathbf{x})\frac{q(\mathbf{x})}{q(\mathbf{x})}d\mathbf{x} = \int f(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}q(\mathbf{x})d\mathbf{x} \\ &= \mathbb{E}_q\left[f(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}\right]\end{aligned}$$

If we choose q in a way that we can easily sample from it, we can approximate this last expectation by Monte Carlo:

$$E_q\left[f(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}\right] \approx \frac{1}{S} \sum_{s=1}^S f(\mathbf{x}^{(s)})\frac{p(\mathbf{x}^{(s)})}{q(\mathbf{x}^{(s)})}, \quad \mathbf{x}^{(s)} \sim q(\mathbf{x})$$

Importance Sampling

Key idea: Do not throw away all rejected samples, but give them lower weight by rewriting the integral as an expectation under a simpler distribution q (**proposal distribution**):

$$\begin{aligned}\mathbb{E}_p[f(\mathbf{x})] &= \int f(\mathbf{x})p(\mathbf{x})d\mathbf{x} \\ &= \int f(\mathbf{x})p(\mathbf{x})\frac{q(\mathbf{x})}{q(\mathbf{x})}d\mathbf{x} = \int f(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}q(\mathbf{x})d\mathbf{x} \\ &= \mathbb{E}_q\left[f(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}\right]\end{aligned}$$

If we choose q in a way that we can easily sample from it, we can approximate this last expectation by Monte Carlo:

$$E_q\left[f(\mathbf{x})\frac{p(\mathbf{x})}{q(\mathbf{x})}\right] \approx \frac{1}{S} \sum_{s=1}^S f(\mathbf{x}^{(s)})\frac{p(\mathbf{x}^{(s)})}{q(\mathbf{x}^{(s)})} = \frac{1}{S} \sum_{s=1}^S w_s f(\mathbf{x}^{(s)}), \quad \mathbf{x}^{(s)} \sim q(\mathbf{x})$$

Properties

- ▶ Unbiased if $q > 0$ where $p > 0$ and if we can evaluate p

Properties

- ▶ Unbiased if $q > 0$ where $p > 0$ and if we can evaluate p
- ▶ Breaks down if we do not have enough samples (puts nearly all weight on a single sample)
 - ▶▶ **Degeneracy**, see also **Particle Filtering** and **SMC**
(Thrun et al., 2005; Doucet et al., 2000)

Properties

- ▶ Unbiased if $q > 0$ where $p > 0$ and if we can evaluate p
- ▶ Breaks down if we do not have enough samples (puts nearly all weight on a single sample)
 - ▶▶ **Degeneracy**, see also **Particle Filtering** and **SMC**
(Thrun et al., 2005; Doucet et al., 2000)
- ▶ **Many draws** from proposal density q required, especially in high dimensions

Properties

- ▶ Unbiased if $q > 0$ where $p > 0$ and if we can evaluate p
- ▶ Breaks down if we do not have enough samples (puts nearly all weight on a single sample)
 - ▶▶ **Degeneracy**, see also **Particle Filtering** and **SMC**
(Thrun et al., 2005; Doucet et al., 2000)
- ▶ **Many draws** from proposal density q required, especially in high dimensions
- ▶ Requires to be able to evaluate true p . Generalization exists for $\tilde{p}$. This generalization is biased (but consistent).

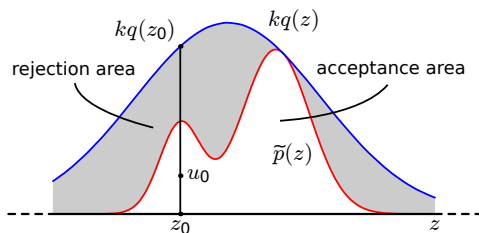
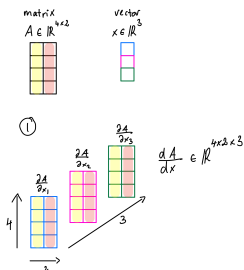
Properties

- ▶ Unbiased if $q > 0$ where $p > 0$ and if we can evaluate p
- ▶ Breaks down if we do not have enough samples (puts nearly all weight on a single sample)
 - ▶▶ **Degeneracy**, see also **Particle Filtering** and **SMC**
(Thrun et al., 2005; Doucet et al., 2000)
- ▶ **Many draws** from proposal density q required, especially in high dimensions
- ▶ Requires to be able to evaluate true p . Generalization exists for $\tilde{p}$. This generalization is biased (but consistent).
- ▶ Does not scale to interesting (high-dimensional) problems

Properties

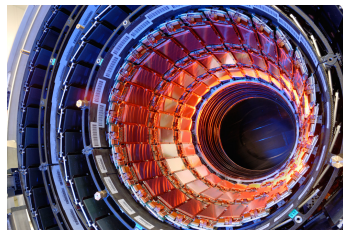
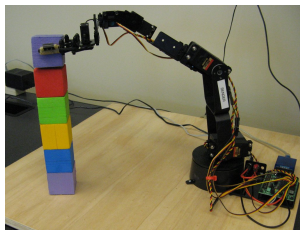
- ▶ Unbiased if $q > 0$ where $p > 0$ and if we can evaluate p
- ▶ Breaks down if we do not have enough samples (puts nearly all weight on a single sample)
 - ▶ **Degeneracy**, see also **Particle Filtering** and **SMC**
(Thrun et al., 2005; Doucet et al., 2000)
- ▶ **Many draws** from proposal density q required, especially in high dimensions
- ▶ Requires to be able to evaluate true p . Generalization exists for $\tilde{p}$. This generalization is biased (but consistent).
- ▶ Does not scale to interesting (high-dimensional) problems
- ▶ Different approach to sample from complicated (high-dimensional) distributions: **Markov Chain Monte Carlo** (e.g., Gilks et al., 1996)

Summary



- Two mathematical challenges in machine learning
 - Differentiation** for optimizing parameters of machine learning models
 - **Vector calculus** and **chain rule**
 - Integration** for computing statistics (e.g., means, variances) and as a principled way to address overfitting issue
 - **Monte-Carlo integration** to solve intractable integrals

Some Application Areas



- ▶ **Image/speech/text/language processing** using deep neural networks (e.g., Krizhevsky et al., 2012 or overview in Goodfellow et al., 2016)
- ▶ **Data-efficient reinforcement learning and robot learning** using Gaussian processes (e.g., Deisenroth & Rasmussen, 2011)
- ▶ **High-energy physics** using deep neural networks or Gaussian processes (e.g., Sadowski et al. 2014; Bertone et al., 2016)

References I

- [1] G. Bertone, M. P. Deisenroth, J. S. Kim, S. Liem, R. R. de Austri, and M. Welling. Accelerating the BSM Interpretation of LHC Data with Machine Learning. arXiv preprint arXiv:1611.02704, 2016.
- [2] C. M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer-Verlag, 2006.
- [3] M. P. Deisenroth, D. Fox, and C. E. Rasmussen. Gaussian Processes for Data-Efficient Learning in Robotics and Control. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 37(2):408–423, Feb. 2015.
- [4] M. P. Deisenroth and C. E. Rasmussen. PILCO: A Model-Based and Data-Efficient Approach to Policy Search. In *Proceedings of the International Conference on Machine Learning*, pages 465–472. ACM, June 2011.
- [5] A. Doucet, S. J. Godsill, and C. Andrieu. On Sequential Monte Carlo Sampling Methods for Bayesian Filtering. *Statistics and Computing*, 10:197–208, 2000.
- [6] W. R. Gilks, S. Richardson, and D. J. Spiegelhalter, editors. *Markov Chain Monte Carlo in Practice: Interdisciplinary Statistics*. Chapman & Hall, 1996.
- [7] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT press, 2016.
- [8] M. I. Jordan, Z. Ghahramani, T. S. Jaakkola, and L. K. Saul. An Introduction to Variational Methods for Graphical Models. *Machine Learning*, 37:183–233, 1999.
- [9] S. Kamthe and M. P. Deisenroth. Data-Efficient Reinforcement Learning with Probabilistic Model Predictive Control. *arXiv:1706.06491*, abs/1706.06491, 2017.
- [10] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet Classification with Deep Convolutional Neural Networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.
- [11] T. P. Minka. *A Family of Algorithms for Approximate Bayesian Inference*. PhD thesis, Massachusetts Institute of Technology, Cambridge, MA, USA, Jan. 2001.
- [12] R. M. Neal. *Bayesian Learning for Neural Networks*. PhD thesis, Department of Computer Science, University of Toronto, 1996.
- [13] A. O’Hagan. Monte Carlo is Fundamentally Unsound. *The Statistician*, 36(2/3):247–249, 1987.

References II

- [14] A. O'Hagan. Bayes-Hermite Quadrature. *Journal of Statistical Planning and Inference*, 29:245–260, 1991.
- [15] M. Opper and O. Winther. Adaptive and Self-averaging Thouless-Anderson-Palmer Mean-field Theory for Probabilistic Modeling. *Physical Review E: Statistical, Nonlinear, and Soft Matter Physics*, 64:056131, 2001.
- [16] C. E. Rasmussen and Z. Ghahramani. Bayesian Monte Carlo. In S. Becker, S. Thrun, and K. Obermayer, editors, *Advances in Neural Information Processing Systems 15*, pages 489–496. The MIT Press, Cambridge, MA, USA, 2003.
- [17] C. P. Robert and G. Casella. *Monte Carlo Methods*. Wiley Online Library, 2004.
- [18] P. Sadowski, J. Collado, D. Whiteson, and P. Baldi. Deep Learning, Dark Knowledge, and Dark Matter. In *NIPS Workshop on High-energy Physics and Machine Learning*, pages 81–87, 2014.
- [19] S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. The MIT Press, Cambridge, MA, USA, 2005.